UET NOWSHERA | ELECTRICAL ENGINEERING

LAB REPORT 08

TWO-DIMENSIONAL ARRAYS IN C++

COURSE: Computer Programming

NAME: Adnan Ahmad

REG. NO: BF25NWELE0744

1. OBJECTIVE

- To understand the concept and structure of a two-dimensional array in C++.
- To learn how to declare and initialize 2D arrays using row-column notation.
- To access and display individual elements of a 2D array using nested loops.
- To perform computations on 2D arrays including row sums, column sums, and finding the maximum value.

2. THEORETICAL BACKGROUND

What is a Two-Dimensional Array? A two-dimensional array is the simplest form of a multidimensional array in C++. It can be visualized as a table with rows and columns, where each element is identified by

two indices: the row index and the column index. In memory, all elements are stored contiguously, with each row following the previous one.

Declaration & Initialization. A two-dimensional array is declared by specifying the data type, name, number of rows, and number of columns, such as int a[3][4];. A 2D array can be initialized at the time of declaration using nested curly braces, where each inner set of braces represents one row. The nested braces are optional; a flat list of values fills the array row by row.

Accessing 2D Array Elements. Each element is accessed using two subscripts: the row index followed by the column index. Nested for loops are typically used to traverse all elements, with the outer loop iterating over rows and the inner loop iterating over columns.

3. SOFTWARE USED

- IDE: DEV-C++ IDE
- Compiler: C++ Compiler (bundled with DEV-C++)

4. EXPERIMENTAL TASKS

Task 1: 2D Array Declaration, Initialization, and Display

A 3x3 integer array named matrix was declared and initialized with the values 1 through 9. Nested for loops were used to iterate through every row and column and display all nine elements in a formatted grid layout. The implementation is as follows:

```
#include "str"><iostream>
using namespace std;

int main() {
    // 3x3 matrix initialized with values 1 to 9
    int matrix[3][3] = {
        {1, 2, 3},
        {4, 5, 6},
        {7, 8, 9}
    };

    // display all elements row by row
    for (int i = 0; i < 3; i++) {
        for (int j = 0; j < 3; j++) {
            cout << matrix[i][j] << "\t"; // tab for column spacing
        }
        cout << endl;
    }
    return 0;
}
```

Task 2: Sum of Rows and Columns in a 2D Array

A 3x3 integer array was declared and initialized with values. The program calculates and displays the sum of each row and each column separately. The total sum of all elements in the array is also computed and displayed at the end. The implementation is as follows:

```
#include "str"><iostream>
using namespace std;

int main() {
    int a[3][3] = {
        {1, 3, 5},
        {2, 4, 6},
        {7, 8, 9}
    };

    int total = 0;

    // compute and display sum of each row
    for (int i = 0; i < 3; i++) {
        int rowSum = 0;
        for (int j = 0; j < 3; j++) {
            rowSum += a[i][j];
        }
        cout << "Row " << i << " sum = " << rowSum << endl;
        total += rowSum; // add row total to grand total
    }

    // compute and display sum of each column
    for (int j = 0; j < 3; j++) {
        int colSum = 0;
        for (int i = 0; i < 3; i++) {
            colSum += a[i][j];
        }
        cout << "Column " << j << " sum = " << colSum << endl;
    }

    cout << "Total sum = " << total << endl;
    return 0;
}
```

Task 3: Finding the Maximum Value in a 2D Array

A 4x4 integer array was declared and initialized with a set of values. The program uses nested for loops to scan every element in the array, tracking the largest value encountered. After traversing all 16 elements, the maximum value is displayed. The implementation is as follows:

```cpp
#include "str"><iostream>
using namespace std;

int main() {
    int grid[4][4] = {
        {3, 17, 8, 2},
        {45, 6, 23, 11},
        {9, 31, 4, 56},
        {12, 7, 38, 19}
    };

    int maxVal = grid[0][0]; // start with the first element

    // scan every element to find the largest
    for (int i = 0; i < 4; i++) {
        for (int j = 0; j < 4; j++) {
            if (grid[i][j] > maxVal) {
                maxVal = grid[i][j]; // update if larger value found
            }
        }
    }

    cout << "Maximum value = " << maxVal << endl;
    return 0;
}
```

5. CONCLUSION

This lab provided hands-on experience with two-dimensional arrays in C++. The rowcolumn structure of 2D arrays was explored through declaration, initialization, and element-by-element display using nested for loops. Practical computations including row sums, column sums, total sum, and maximum value search were implemented, demonstrating how nested iteration is applied to process grid-structured data. Twodimensional arrays are a foundational concept in matrix operations, image processing, and table-driven programs encountered in later stages of the degree.